

PROBING TRANSFORMER REASONING TRANSFER THROUGH ALPHABET EQUIVALENCE

Anonymous authors

Paper under double-blind review

ABSTRACT

A reasoner is *abstract* if it applies a general procedure to a problem’s structure, indifferent to how that structure is concretely instantiated; it is *concrete* if what it learned is tied to the particular instantiation trained on. This gives a testable marker: an abstract reasoner should transfer mastery of one problem instance to another. We study equivalence via alphabet renaming, and test transfer using a weight-transplant control – copying trained per-symbol weights onto the new instance, no retraining – distinguishing genuine reasoning failure from symbols never trained on. We instantiate this with Sudoku, where an equivalent puzzle is any relabeling of its digits. A transformer trained on one such puzzle transfers to an equivalent one no better than an untrained network – yet the transplant control recovers its original performance exactly. Reasoning, in that narrow sense, is abstract; recognizing that a new instance calls for it is not. We probe this recognition gap three ways. Freezing the shared network and adapting only a new alphabet’s parameters fares worse than training an unfrozen model from scratch on the same data: prior exposure to other alphabets does not ease learning a new one. Recombining already-known symbols into unfamiliar combinations yields graded, partial success – above chance, short of the seamless transfer an abstract reasoner should show, degrading as combinations diverge from training. Replacing the model’s fixed, per-alphabet output vocabulary with a mechanism that selects among a puzzle’s own visible symbols helps substantially with recombination, but only marginally with transfer to a truly unseen alphabet. These results isolate a specific failure to recognize alphabet-equivalent instances as such, distinct from a general failure to reason, and show several natural interventions narrow, but do not close, this gap.

1 INTRODUCTION

A model that has learned to solve a problem can be said to reason *abstractly* about it if the procedure it applies operates on the problem’s structure, indifferent to how that structure happens to be dressed up in a particular presentation. It reasons only *concretely* if what it has learned is tied to the specific surface form it was trained on, rather than to the structure underneath. This distinction is central to any claim that a learned system reasons rather than merely fits patterns in its training distribution, but such claims are frequently made without a test that could actually distinguish the two possibilities.

We propose a concrete, checkable operationalization of this distinction. An abstract reasoner should treat two different instances of the same underlying problem interchangeably: having mastered one, it should transfer to solving the other, since its procedure does not depend on which particular instantiation it happens to face. We instantiate “different instances of the same problem” through the simplest transform that leaves a problem’s structure completely intact while changing its surface form entirely: renaming its symbol alphabet. Two problems P and Q are *equivalent* under this definition when Q is obtained from P purely by relabeling which symbols represent which underlying roles – nothing about the structure, constraints, or solution changes, only the names used to present it.

This gives more than a single empirical test: it gives a general recipe applicable to any symbolic or combinatorial domain with an analogous notion of relabeling. The recipe has two parts. First, define equivalence via a checkable relabeling, and measure whether a model trained on one instance

transfers, unaided, to an equivalent one. Second – and this is the part that is easy to get wrong – use a targeted control to separate two failure modes that a naive transfer evaluation cannot distinguish on its own. A model may fail on a new instance either because its underlying reasoning does not generalize, or simply because the new instance’s symbols were never seen during training, so any parameters associated with them are untrained noise regardless of how good the model’s reasoning otherwise is. We isolate the second explanation with a *weight-transplant* control: after training, we copy the parameters associated with each of P ’s symbols directly onto the parameters for Q ’s corresponding symbols, using the known-by-construction correspondence, with no further training. This tells us how the model would perform on Q if it already knew the correspondence between the two instances’ symbols – precisely the information a naive zero-shot evaluation cannot provide on its own.

We instantiate this recipe concretely using Sudoku: a well-defined, verifiable, and cheaply generated combinatorial puzzle whose solution is invariant to any consistent renaming of its 9 symbols, making it a clean testbed for this question. Training a transformer to solve Sudoku puzzles written in one symbol alphabet, we find that it transfers to an equivalent puzzle in a different alphabet at chance level – performance indistinguishable from a network that was never trained at all – even though the weight-transplant control recovers its original accuracy *exactly*. This is the paper’s central result: the model’s procedure does reuse across equivalent instances once it is told how their symbols correspond; what it lacks is any mechanism for discovering that correspondence on its own. We then probe this recognition gap along three further axes – pretraining diversity, an architectural change that removes a specific structural bottleneck, and training-time data diversity at a much larger scale – and show that each narrows, but does not close, the gap, with a further finding that training diversity resolves the gap into two distinct sub-problems with different answers: recombining already-known symbols is solved by sufficient diversity, while transfer to a genuinely novel symbol is not.

Contributions.

- An operationalized, reusable methodology for testing knowledge transfer between equivalent instances of a symbolic problem, including a weight-transplant control that isolates genuine reasoning-transfer failure from the confound of never-trained parameters.
- Applied to Sudoku, we show naive transfer across alphabet-equivalent puzzles fails completely – statistically indistinguishable from an untrained network – while the transplant control recovers full performance exactly, demonstrating the failure is one of recognition, not of reusable computation (Section 4.1).
- We show this recognition gap persists across two further natural interventions: pretraining diversity across several known alphabets, and a redesigned architecture that removes a specific structural bottleneck that prevents zero-shot generalization by construction – each narrows but does not close the gap (Section 4.2–3).
- We show training-time data diversity resolves this gap into two distinct sub-problems with different answers: recombining already-known symbols in novel combinations is essentially solved by sufficient training diversity, while transfer to a genuinely novel, never-seen symbol remains unsolved by every intervention tested (Section 4.4).

108	2	RELATED WORK
109		
110	3	PROBLEM SETUP AND METHOD
111		
112	4	EXPERIMENTS AND RESULTS
113		
114	5	DISCUSSION
115		
116	6	LIMITATIONS
117		
118	7	CONCLUSION
119		
120		
121		REFERENCES
122		
123		
124		
125		
126		
127		
128		
129		
130		
131		
132		
133		
134		
135		
136		
137		
138		
139		
140		
141		
142		
143		
144		
145		
146		
147		
148		
149		
150		
151		
152		
153		
154		
155		
156		
157		
158		
159		
160		
161		